

BUILD_MASTER

NetworkPeer — Master Architecture, Build & Operations Manual

1. System Architecture Overview

NetworkPeer is an anonymous-until-accepted decentralized field verification marketplace connecting **Clients** who post verifiable physical tasks (inspections, audits, photography, mystery shopping) with nearby verified **Workers** who collect cryptographically verifiable evidence.

```
flowchart TD
    subgraph Clients["Frontend Clients"]
        WEB["TanStack Start Web App (Vercel)"]
        ANDROID["Native Android App (Jetpack Compose)"]
        EXPO["Cross-Platform Mobile (Expo / React Native)"]
    end

    subgraph Ingress["Ingress & Proxy"]
        VPROXY["Vercel Edge Proxy / Nitro Handler"]
        ALB["AWS Application Load Balancer"]
    end

    subgraph Compute["AWS ECS Fargate Cluster"]
        API["Fastify / Node.js API Service (Port 3000, 10.0.2.11)"]
        WORKER["Background Processing Worker Service"]
        LAMBDA["Cognito Custom Auth Lambda (SMS OTP Dispatch via SNS)"]
    end

    subgraph Data["AWS Staging Data Stores"]
        RDS["AWS RDS PostgreSQL 16.15 + PostGIS"]
        COG["AWS Cognito User Pool"]
        S3["AWS S3 Media Bucket"]
    end

    WEB -->|Same-Origin HTTPS| VPROXY
    VPROXY -->|HTTP Backend Ingress| ALB
```

```

ANDROID -->|HTTP Direct API| ALB
EXPO -->|HTTP Direct API| ALB

ALB --> API
API --> RDS
API --> COG
API --> S3
COG -->|Custom Auth Triggers| LAMBDA
WORKER --> RDS
WORKER --> S3

```

2. Infrastructure Details & Verified Endpoints

2.1 AWS Resource Inventory (Account 872118049328 , Region eu-north-1)

Component	AWS Resource Identifier	Configuration / Endpoint	Status
Application Load Balancer	networkpeer-staging-api-alb	http://networkpeer-staging-api-alb-969746120.eu-north-1.elb.amazonaws.com	Active / Target Healthy
ECS Cluster	networkpeer-staging-cluster	Fargate launch type	Active
ECS API Service	networkpeer-staging-api	Task Definition networkpeer-staging-api:6	1/1 Running Healthy
RDS PostgreSQL + PostGIS	networkpeer-staging-postgis	networkpeer-staging-postgis.c7e6uqswai46.eu-north-1.rds.amazonaws.com:5432	Active / Migration Applied
Cognito User Pool	networkpeer-staging-users	eu-north-1_GKLq1Fqzz	Active
Cognito App Client	networkpeer-staging-api	45mgqp0jff5so65daesm3g4e1u (CUSTOM_AUTH enabled)	Active
Cognito Custom Auth Lambda	networkpeer-staging-cognito-custom-auth	Node.js 20.x, SNS SMS OTP dispatch	Active
S3 Media Bucket	networkpeer-media-staging-trivaro	Private media storage with KMS encryption	Active

Component	AWS Resource Identifier	Configuration / Endpoint	Status
S3 Terraform State	networkpeer-terraform-state	Remote infrastructure lock	Active

3. Step-by-Step Local Development Setup (Mac M1)

3.1 Prerequisites

Ensure the following tools are installed on your Mac M1: - **Node.js:** v20.x or v22.x (or v23.x) - **Java JDK:** OpenJDK 17 (/opt/homebrew/opt/openjdk@17) - **Android SDK:** Located at ~/Library/Android/sdk - **AWS CLI v2:** Installed and authenticated

```
# Add to ~/.zshrc or ~/.bash_profile
export JAVA_HOME="/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home"
export ANDROID_HOME="$HOME/Library/Android/sdk"
export PATH="$JAVA_HOME/bin:$ANDROID_HOME/platform-tools:$ANDROID_HOME/cmdline-tools/latest/bin:$PATH"
export AWS_ACCESS_KEY_ID="AKIA4WDSJLIYCJXLNKOM"
export AWS_SECRET_ACCESS_KEY="<YOUR_SECRET_KEY>"
export AWS_DEFAULT_REGION="eu-north-1"
```

3.2 Running the Web Application Locally

```
cd NetworkPeer-platform-main

# Install dependencies
npm install

# Build contracts first
npm --prefix ../packages/contracts run build

# Start the development server (runs on http://localhost:5173)
npm run dev
```

4. Database Schema & Inspection Protocol

4.1 Schema Overview (PostgreSQL 16.15 with PostGIS)

All 41 database schema migrations have been applied. Key tables:

- 1. **users** : System identity accounts (id , phone_number , role , is_active , is_verified , created_at).
- 2. **jobs** : Job postings (id , client_id , worker_id , title , description , category , status , budget_cents , currency , location PostGIS Point, address , scheduled_at , escrow_status).

3. **job_subtasks** : Individual evidence steps (`id` , `job_id` , `title` , `description` , `sequence_order` , `is_required` , `status`).
4. **job_subtask_media** : Cryptographic evidence records (`id` , `subtask_id` , `worker_id` , `s3_key` , `s3_bucket` , `checksum_sha256` , `location` , `status`).
5. **wallet_ledger** : Double-entry ledger balances for Clients and Workers.
6. **ledger_transactions** : Individual escrow holds, releases, fees, and payouts.
7. **payment_operations** : Idempotent payment transactions and Razorpay/Stripe tracking.
8. **media_processing_outbox** : Async pipeline for EXIF extraction, hashing, and fraud detection.

4.2 Querying and Managing the Database

Because the RDS instance resides in a private VPC subnet for enterprise security, run queries through an authorized ECS task or bastion:

```
# Run one-off SQL query via ECS Fargate task:
aws ecs run-task \
  --cluster networkpeer-staging-cluster \
  --task-definition networkpeer-staging-api:6 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[\"subnet-0b9ee3f044bb48530\"],securityGroups=[\"sg-06fcbe5f356bf474c\"],assignPublicIp=\"ENABLED\"}" \
  --overrides '{"containerOverrides":[{"name":"api","command":["node","-e","const { Pool } = require(\"pg\");const pool = new Pool({ connectionString: process.env.DATABASE_URL });pool.query(\"SELECT id, phone_number, role, created_at FROM users ORDER BY created_at DESC LIMIT 10;\", (err, res) => {if (err) console.error(err);else console.table(res.rows);pool.end();});}]]}' \
  --region eu-north-1
```

5. S3 Media Storage Management Protocol

5.1 Stored Evidence Structure

Files uploaded by Workers are stored under:

```
s3://networkpeer-media-staging-trivaro/jobs/{jobId}/evidence/{subtaskId}/{evidenceId}_{timestamp}.j
```

5.2 S3 Management Commands

```
# 1. List all evidence files in bucket
aws s3 ls s3://networkpeer-media-staging-trivaro/ --recursive --human-readable --summarize
```

```
# 2. Inspect a specific job's media
aws s3 ls s3://networkpeer-media-staging-trivaro/jobs/<JOB_ID>/ -
    -recursive

# 3. Generate a pre-signed download URL for private review (valid
    for 15 minutes)
aws s3 presign s3://networkpeer-media-staging-
    trivaro/<OBJECT_KEY> --expires-in 900

# 4. Clean up test uploads or orphaned files
aws s3 rm s3://networkpeer-media-staging-trivaro/test/ --
    recursive
```

6. Android Physical Device Testing Runbook (Mac M1)

6.1 APK Location & Metadata

- **Path:** apps/android/app/build/outputs/apk/development/debug/app-development-debug.apk
- **Application ID:** com.networkpeer.mobile.dev
- **Activity:** com.networkpeer.mobile.MainActivity
- **Size:** 34.6 MB
- **Target Backend:** http://networkpeer-staging-api-alb-969746120.eu-north-1.elb.amazonaws.com/api/v1

6.2 Step-by-Step Installation via Data Cable

Step 1: Enable USB Debugging on Android Phone

1. Open **Settings** on the Android phone.
2. Go to **About Phone** and tap **Build Number** 7 times until you see "You are now a developer!".
3. Navigate to **System** -> **Developer Options**.
4. Enable **USB Debugging** and toggle on **Install via USB**.

Step 2: Connect Phone to Mac M1

1. Plug the phone into your Mac M1 using a USB-C data cable.
2. When the prompt "Allow USB debugging?" appears on the phone screen, check "Always allow from this computer" and tap **Allow**.

Step 3: Run ADB Commands

In your Mac Terminal:

```
# 1. Verify ADB detects the phone
/Users/adityasharma/Library/Android/sdk/platform-tools/adb
    devices -l

# 2. Install the APK onto the phone
/Users/adityasharma/Library/Android/sdk/platform-tools/adb install -r
    /Users/adityasharma/Desktop/NETWORKPEER/apps/android/app/build/outputs/apk/development/debug/app-
    development-debug.apk

# 3. Launch the application
```

```

/Users/adityasharma/Library/Android/sdk/platform-tools/adb shell am
start -n
com.networkpeer.mobile.dev/com.networkpeer.mobile.MainActivity

# 4. Stream real-time logs filtered for NetworkPeer
/Users/adityasharma/Library/Android/sdk/platform-tools/adb logcat
-s NetworkPeer:*

```

6.3 Troubleshooting Common Hardware-Link Issues

- **adb devices shows unauthorized** : Unplug the USB cable, run `adb kill-server && adb start-server`, plug the cable back in, and accept the authorization prompt on the phone screen.
- **adb devices is empty**:
 1. Ensure the USB cable is a **data cable** (not a charging-only cable).
 2. In your phone's notification shade, tap the USB options and switch from *"Charging only"* to *"File Transfer (MTP)"*.
 3. Try connecting via a different USB-C port on your Mac.

7. Frontend Deployment & API Reverse Proxy

7.1 The Mixed Content & Map Solution

- **Zero Mixed Content**: Browsers prohibit `https://` websites from requesting `http://` backend endpoints. TanStack Start's `server.ts` now reverse-proxies `/api/v1/*` to the AWS ALB. The browser communicates exclusively via HTTPS (`https://networkpeer-platform.vercel.app/api/v1/...`).
- **Watermark-Free Maps**: In `LocationPicker.tsx`, the Carto tile URL was replaced with standard OpenStreetMap tiles (`https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`) styled with clean dark CSS inversion filters, completely eliminating the *"API KEY REQUIRED"* watermark.
- **Vipul Post-a-Job**: The unified job posting form with interactive location picker, dynamic evidence checklist, anonymized preview, and budget calculation is fully integrated and operational.